# Probleme de concurență
## *- o introducere -*

# Sistem concurent

Un sistem software concurent este format din mai multe entități care rulează în paralel (procese, fire de execuție)

Pentru simplificare, vom folosi conceptul de fir de execuție (thread) pentru a desemna și exemplifica o astfel de entitate (termen consacrat)

Firele de execuție sunt parte a aceluiași sistem => colaborează între ele, deoarece au scopuri comune

# Condiție de cursă (race condition)

Thread-urile rulează în paralel și comunică între ele
=> pot să apară situații de interacțiune complexe și imprevizibile.

**Condiție de cursă (race condition)** = *o condiție care apare atunci când corectitudinea execuției unui program depinde de ordinea relativă în timp a unor evenimente multiple, cum ar fi execuția diverselor operații efectuate de thread-uri*

Rezultatul poate, deci, depinde de modul în care firele de execuție sunt planificate și rulează.

# Conflict de date (data race)

*Conflictul de date (data race)* este una dintre cele mai importante surse de condiții de cursă (dar nu e singura)

*Data race* apare când toate următoarele condiții sunt îndeplinite:
- 2 sau mai multe thread-uri accesează aceeași zonă conținând date partajate (= zonă de memorie, variabilă etc.)
- cel puțin una din operații modifică / scrie în zona respectivă
- thread-urile nu iau măsuri pentru a-și sincroniza accesul la acele date

Alte surse de condiții de cursă: accesul la fișiere, apeluri de funcții sistem și alte operații pentru care contează ordinea

# Exemplu de data race

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NTHREADS 10
#define NOPS 10000

long counter = 0; // The shared
variable

// Thread body
void *increment(void *arg)
{
    for (int i = 0; i < NOPS; i++)
    {
        counter++;
    }
    return NULL;
}
```

```
int main() {
    pthread_t threads[NTHREADS];
    int i;

    // Create threads
    for (i = 0; i < NTHREADS; i++)
    {
        if (pthread_create(&threads[i], NULL, increment, NULL) != 0)
            { perror("Error creating thread"); return 1; }
    }

    // Wait for all threads to end
    for (i = 0; i < NTHREADS; i++)
    {
        if (pthread_join(threads[i], NULL) != 0)
            { perror("Error joining thread"); return 1; }
    }

    printf("Expected final counter: %d\n", NTHREADS * NOPS);
    printf("Real final counter:   %ld\n", counter);

    return 0;
}
```

# Exemplu de rulare a programului

```
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ gcc -o datarace -pthread datarace.c
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace
Expected final counter: 100000
Real final counter:   61572
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace
Expected final counter: 100000
Real final counter:   27115
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace
Expected final counter: 100000
Real final counter:   98256
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace
Expected final counter: 100000
Real final counter:   100000
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace
Expected final counter: 100000
Real final counter:   100000
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace
Expected final counter: 100000
Real final counter:   100000
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace
Expected final counter: 100000
Real final counter:   59357
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace
Expected final counter: 100000
Real final counter:   36658
```

# Explicație

Problema:

- firele de execuție ajung la incrementarea contorului la momente arbitrare de timp: apar suprapuneri (ajung “deodată”)
- operația de incrementare se face în mai mulți pași, durează
- în funcție de modul în care multiple thread-uri se suprapun *complet sau parțial* în această operație, rezultatul poate fi diferit

Soluția:

- firele trebuie să își *sincronizeze* între ele execuția pe durata porțiunii de cod “cu probleme” (numită *regiune critică*): doar un thread să facă la un moment dat incrementarea

# Sincronizare

Există diverse primitive pe care sistemul de operare sau platforma software le oferă programelor pentru sincronizarea thread-urilor:

- mutex
- semafoare
- monitoare
- lacăte
- bariere

...

# Mutex (conceptul general)

Originea numelui: *mutual exclusion (excludere reciprocă)*

- un *mutex* este o resursă accesibilă thread-urilor, care poate fi creată prin program; pot fi create mai multe mutex-uri, după nevoie
- pe mutex sunt definite cel puțin 2 operații: *lock*, *unlock*
- pot exista și operații mai avansate: *trylock*, *timedlock* etc.
- un thread poate cere *blocarea* mutex-ului *(lock)*
- doar un singur thread poate ține blocat mutex-ul la un moment dat
- când dorește, thread-ul care are mutex-ul poate să îl deblocheze *(unlock)*
- dacă un thread apelează *lock* pe un mutex deja blocat, va fi pus în așteptare (operația *lock* rămâne blocată, nu se întoarce, thread-ul nu continuă execuția ci așteaptă deblocarea mutex-ului)
- când un thread apelează unlock, unul dintre firele care așteptau deblocarea mutex-ului e “eliberat” (continuă execuția după apelul *lock*) și preia mutex-ul

# Mutex (UNIX)

Disponibil în biblioteca pthreads (POSIX Threads), ca tipul de date `pthread_mutex_t`

Inițializare:

```
pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
```

Blocare (Lock):

```
pthread_mutex_lock(pthread_mutex_t *mutex);
```

Deblocare (Unlock):

```
pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Distrugere:

```
pthread_mutex_destroy(pthread_mutex_t *mutex);
```

# Rezolvarea exemplului, folosind mutex

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#define NTHREADS 10
#define NOPS 10000
long counter = 0; // The shared variable
pthread_mutex_t mutex;

// Thread body
void *increment(void *arg)
{
    for (int i = 0; i < NOPS; i++)
    {
        if (pthread_mutex_lock(&mutex) != 0)
            { perror("Error locking mutex"); pthread_exit(NULL); }

        counter++;

        if (pthread_mutex_unlock(&mutex) != 0)
            { perror("Error unlocking mutex"); pthread_exit(NULL); }
    }
    return NULL;
}
```

# Rezolvarea exemplului, folosind mutex

```
int main()
{
    pthread_t threads[NTHREADS];
    int i;

    if (pthread_mutex_init(&mutex, NULL) != 0)
        { perror("Error initializing mutex"); }

   for (i = 0; i < NTHREADS; i++) {
        if (pthread_create(&threads[i], NULL, increment, NULL) != 0)
            {perror("Error creating thread"); return 1;}
    }

    for (i = 0; i < NTHREADS; i++) {
        if (pthread_join(threads[i], NULL) != 0)
            { perror("Error joining thread"); return 1; }
    }

    if (pthread_mutex_destroy(&mutex) != 0)
        { perror("Error destroying mutex"); }
    printf("Expected final counter: %d\n", NTHREADS * NOPS);
    printf("Real final counter:   %ld\n", counter);
    return 0;
}
```

# Exemplu de rulare a programului

```
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ gcc -o datarace-fixed -pthread datarace-fixed.c
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace-fixed
Expected final counter: 100000
Real final counter:   100000
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace-fixed
Expected final counter: 100000
Real final counter:   100000
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace-fixed
Expected final counter: 100000
Real final counter:   100000
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace-fixed
Expected final counter: 100000
Real final counter:   100000
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace-fixed
Expected final counter: 100000
Real final counter:   100000
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace-fixed
Expected final counter: 100000
Real final counter:   100000
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace-fixed
Expected final counter: 100000
Real final counter:   100000
danc@vivobook:~/scoala/cursuri/so/ex-mutex$ ./datarace-fixed
Expected final counter: 100000
```